

Análise e Desenvolvimento de Sistemas

PROJETO

Sistema ERP de Estoque e Faturamento

Equipe

Gabriel Medeiros de Mendonça - 2515050024

Professor

Prof. Bruno Rafael Araújo Vasconcelos

Campina Grande - 2026

Sumário

1. Sobre o Projeto
2. Como Funciona o Sistema
 - 2.1 Arquitetura
 - 2.2 Fluxo de uso
 - 2.3 Páginas do Sistema
 - 2.4 Regras de Negócio
3. Tecnologias Utilizadas
4. Instalação e Execução
 - 4.1 Pré-requisitos
 - 4.2 Backend
 - 4.3 Frontend
 - 4.4 Dependências
5. Repositórios e Vídeo
 - 5.1 Repositórios no GitHub
 - 5.2 Requisitos do sistema
 - 5.2.1 Priorização de requisitos
6. Referências Bibliográficas

1. Sobre o Projeto

O KorpERP é um sistema ERP web voltado à gestão de produtos em estoque e emissão de notas fiscais. O projeto é dividido em dois contextos principais: Estoque, responsável pelo cadastro, consulta e débito de produtos; e Faturamento, responsável pela criação, listagem e impressão de notas fiscais.

O sistema foi desenvolvido com front-end em Angular e back-end em .NET 8, utilizando SQL Server para persistência e RabbitMQ para integração assíncrona entre os contextos. A solução aplica Clean Architecture, DDD, APIs REST, paginação, tratamento centralizado de erros, concorrência otimista e idempotência no processamento de mensagens.

Objetivo geral: entregar uma aplicação administrativa integrada, capaz de controlar estoque e faturamento com separação clara de responsabilidades entre interface, regras de aplicação, domínio e infraestrutura.

2. Como Funciona o Sistema

2.1 Arquitetura

O sistema é dividido em três blocos executáveis principais:

- Frontend Angular: aplicação acessada pelo navegador, servida via Nginx em Docker ou via ng serve em ambiente local.
- Estoque API: API REST em .NET 8 para cadastro, consulta, paginação e débito de produtos.
- Faturamento API: API REST em .NET 8 para criação, consulta, paginação e impressão de notas fiscais.

As APIs usam SQL Server como banco de dados. O Faturamento consulta o Estoque de forma síncrona para validar produtos e saldo antes de fechar uma nota. Após a impressão, o Faturamento publica o evento nota-impressa no RabbitMQ; a API de Estoque consome esse evento e debita o saldo de forma assíncrona.

A organização do back-end segue camadas de Domain, Application, Infra.Data, Infra.ioC e API. O domínio concentra entidades e regras de negócio; a aplicação concentra DTOs, serviços, interfaces e mapeadores; a infraestrutura implementa banco, repositórios, clientes HTTP e mensageria; e as APIs expõem os endpoints REST.

2.2 Fluxo de uso

1. O usuário acessa o front-end em <http://localhost:4200>.
2. A aplicação exibe o menu lateral com as áreas de Produtos e Notas Fiscais.
3. Em Produtos, o usuário lista produtos paginados e cadastra novos itens com código, descrição e saldo inicial.
4. Em Notas Fiscais, o usuário lista notas paginadas e cadastra uma nota com um ou mais produtos.

5. Ao criar a nota, a API de Faturamento busca o próximo número sequencial e valida os produtos no Estoque.
6. Ao imprimir a nota, o sistema valida novamente o saldo, fecha a nota e publica um evento no RabbitMQ.
7. O consumidor da API de Estoque processa o evento, evita duplicidade por MessageId e debita o saldo dos produtos.

2.3 Páginas do Sistema

Produtos:

- Tela de listagem paginada com código, descrição e saldo em estoque.
- Modal de cadastro de produto com validação dos campos obrigatórios.

Notas Fiscais:

- Tela de listagem paginada com número, data, produtos vinculados e status Aberta ou Fechada.
- Modal de cadastro de nota fiscal, permitindo selecionar produtos e informar quantidades.
- Ação de impressão disponível para notas em aberto.

Navegação:

- Menu lateral compartilhado para acesso às rotas /produtos e /notas-fiscais.

2.4 Regras de Negócio

- Produto deve possuir código e descrição preenchidos.
- Produto não pode ser criado com saldo negativo.
- Débito de estoque exige quantidade maior que zero.
- O estoque não pode ficar negativo; caso o saldo seja insuficiente, a operação é rejeitada.
- Nota fiscal é criada inicialmente com status Aberta.
- Nota fiscal recebe número sequencial gerado pelo repositório de Faturamento.
- Nota fechada não pode ser alterada ou impressa novamente.
- Antes da criação e da impressão, os itens da nota são validados contra a API de Estoque.
- A impressão fecha a nota e dispara o evento nota-impressa para débito assíncrono do estoque.
- O consumidor de estoque registra mensagens processadas para garantir idempotência.

- A entidade Produto usa RowVersion para concorrência otimista em atualizações simultâneas.

3. Tecnologias Utilizadas

Tecnologia	Versão	Finalidade
.NET	8.0	Criação das APIs REST de Estoque e Faturamento
ASP.NET Core	8.0	Controllers, middlewares, injeção de dependência e Swagger
Entity Framework Core	8.0.0	ORM, DbContext, migrations e persistência SQL Server
SQL Server	2022 Docker image	Banco de dados das APIs
RabbitMQ	management image / client 7.2.1	Mensageria assíncrona entre Faturamento e Estoque
Polly	8.0.0	Retry e circuit breaker nas chamadas HTTP
Angular	19.2.x	Aplicação front-end SPA/SSR
Angular Material	19.2.19	Componentes visuais, tabela, paginador, dialogs e snackbar
RxJS	~7.8.0	Fluxos reativos das chamadas HTTP
TypeScript	~5.7.2	Linguagem do front-end
Docker / Docker Compose	Atual	Orquestração local dos serviços
Nginx	alpine	Servidor estático do front-end em produção

4. Instalação e Execução

4.1 Pré-requisitos

- Docker Desktop instalado e em execução.
- .NET 8 SDK, caso execute as APIs fora do Docker.
- Node.js 20 ou compatível, caso execute o front-end fora do Docker.
- Portas livres: 1434 para SQL Server, 5672 e 15672 para RabbitMQ, 5001 e 5002 para APIs, 4200 para o front-end.

4.2 Backend

Executar com Docker Compose:

```
cd F:\KorpERP\back-end\KorpERP
```

```
docker compose up --build
```

Serviços disponibilizados:

- SQL Server: localhost,1434
- RabbitMQ Management: <http://localhost:15672>, usuário guest e senha guest
- Estoque API Swagger: <http://localhost:5001/swagger>
- Faturamento API Swagger: <http://localhost:5002/swagger>

Endpoints principais da Estoque API:

Método	Endpoint	Descrição
GET	/api/produtos	Lista produtos paginados
GET	/api/produtos/{codigo}	Busca produto por código
POST	/api/produtos	Cria produto
PATCH	/api/produtos/{codigo}/debitar	Debita saldo de produto

Endpoints principais da Faturamento API:

Método	Endpoint	Descrição
GET	/api/notasfiscais	Lista notas fiscais paginadas
GET	/api/notasfiscais/{id}	Busca nota fiscal por ID
POST	/api/notasfiscais	Cria nota fiscal
POST	/api/notasfiscais/{id}/imprimir	Fecha nota e publica evento de débito de estoque

4.3 Frontend

Executar com Docker Compose:

```
cd F:\KorpERP\front-end\korp-erp
```

```
docker compose up --build
```

A aplicação fica disponível em <http://localhost:4200>. No Docker, as URLs configuradas no environment são /api/estoque e /api/faturamento, permitindo proxy/reverse proxy pelo Nginx.

Execução local sem Docker:

```
cd F:\KorpERP\front-end\korp-erp
```

```
npm install
```

```
npm start
```

4.4 Dependências

Backend:

- Microsoft.EntityFrameworkCore, Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Design
- RabbitMQ.Client
- Swashbuckle.AspNetCore
- Microsoft.Extensions.Http.Polly

Frontend:

- @angular/core, @angular/router, @angular/forms, @angular/material, @angular/cdk
- rxjs, tslib, zone.js

- express e @angular/ssr para suporte SSR

5. Repositórios e Vídeo

5.1 Repositórios no GitHub

Repositório do projeto: https://github.com/gabriel-afd/Korp_Teste_Gabriel_Medeiros

Vídeo de apresentação:

<https://drive.google.com/file/d/1599PdseNbdqzuhXt32JIQINuJCSibgfh/view?usp=sharing>

5.2 Requisitos do sistema

Código	Requisito	Tipo	Status
RF01	Cadastrar produto com código, descrição e saldo inicial	Funcional	Entregue
RF02	Listar produtos com paginação	Funcional	Entregue
RF03	Consultar produto por código	Funcional	Entregue
RF04	Criar nota fiscal com itens de produto	Funcional	Entregue
RF05	Listar notas fiscais com paginação	Funcional	Entregue
RF06	Imprimir nota fiscal e alterar status para Fechada	Funcional	Entregue
RF07	Validar saldo antes da emissão/impressão da nota	Funcional	Entregue
RF08	Debitar estoque após impressão da nota	Funcional	Entregue
RNF01	Separar o back-end por camadas e contextos de domínio	Não funcional	Entregue
RNF02	Usar mensageria assíncrona para integração entre contextos	Não funcional	Entregue
RNF03	Aplicar concorrência otimista e idempotência	Não funcional	Entregue
RNF04	Disponibilizar execução com Docker Compose	Não funcional	Entregue

5.2.1 Priorização de requisitos

Micro entrega	Requisitos priorizados	Entregas realizadas
1	RF01, RF02, RF03	Criação do contexto de Estoque, entidade Produto, repositório, service,

		endpoints REST e tela de produtos.
2	RF04, RF05, RF07	Criação do contexto de Faturamento, entidade NotaFiscal, itens, services, endpoints REST e tela de notas fiscais.
3	RF06, RF08, RNF02	Implementação do fluxo de impressão, publicação do evento nota-impressa e consumer para débito do estoque.
4	RNF01, RNF03, RNF04	Organização em Clean Architecture/DDD, tratamento de erros, RowVersion, idempotência e Docker Compose para execução integrada.

6. Referências Bibliográficas

Microsoft. ASP.NET Core Documentation.
 Microsoft. Entity Framework Core Documentation.
 Microsoft. .NET 8 Documentation.
 Angular Documentation.
 Angular Material Documentation.
 RxJS Documentation.
 RabbitMQ Documentation.
 Polly Documentation.
 Docker Documentation.
 MDN Web Docs - JavaScript e APIs Web.

Campina Grande, Junho de 2026